TUTORIAL SESSIONS

5210COMP:

Software Engineering for Games

Module Leader: Dr Chris Carter

Tutorial 17:

UE5 Tutorial #03

User Interface Elements and Game Scenarios

School of Computer Science and Mathematics

LIVERPOOL JOHN MOORES UNIVERSITY

Tutorial Overview

In today's tutorial we will be finishing our look at the **FiringRange** project. There are still Unreal Engine tutorials, but they will be using a different project, so that you can apply those concepts to the assignment (which are more software engineering based, than UE5 feature related).

In this final look at the FiringRange, we will look at two further topics, which is **User Interface Integration** and **Projectile Bullet customisation and interaction**.

- In **Task 1**, we will customise the Projectile and our Collisions and see how we can work with the in-built tools and damage model of UE5 to provide reactions and player feedback, as well as a further look at event dispatchers.

- In **Task 2**, we will look at the User Interface concepts of the gameplay framework and see how we can associate the User Interface with the Player Controller and Heads Up Display classes.

Task 1: Customising our Projectile Behaviours

This Tutorial is using a copy of the completed Solution to Tutorial 16 as the basis – if you have not attempted Tutorial 16 first, you should do so before starting this tutorial.

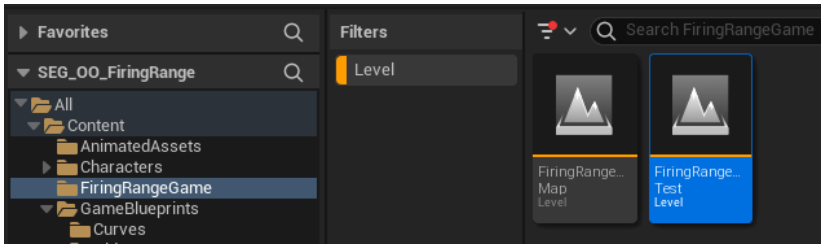
If you do not have a copy of Tutorial 16's solution available, again, you can download the model solution to that and use that as the basis;

Do not use this as a substitute for completing Tutorial 16 – this is an important tutorial that will help you apply several key concepts to the coursework.

Step 1: Setting up the Project

1. Rename the outer folder to **SEG_Tutorial17** and open the .**uproject** to open our UE5 Firing Range Project.

2. Once the project is open, go into the **FiringRangeGame** folder and right-click on FiringRangeMap and Duplicate it – name this duplicate **FiringRangeUITest.**



3. Open this map.

4. Remove any existing Targets from the Level – these should be in the Targets folder; however, they may also be in the Root of the Level (i.e., where the PlayerStart is).

5. Remove the Obstacle Spawning System and Control Units from the scene as well.

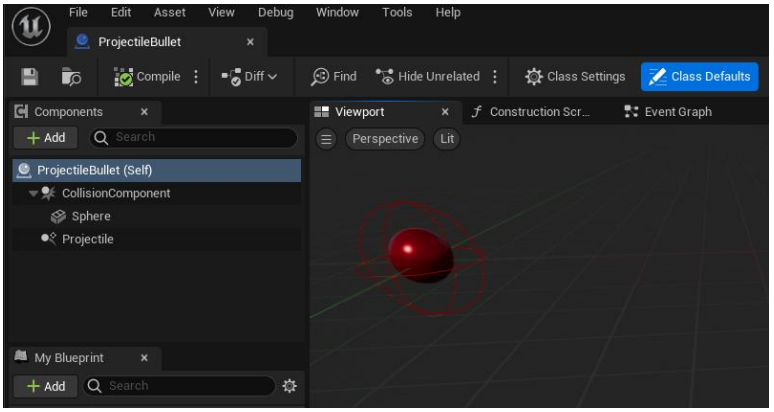
6. Add five instances of our **DT_SnapRot_Y** Target class we completed in Tutorial 16 to the Level, using the technique you feel is the easiest to use, laying them out a row across the level.

7. Switch the Default Pawn Class of our **FiringRangeGameMode** to the **ProjectileCharacter** if it is not already set. Test that when you play the game, it is the Projectile Weapon that is being fired.

8. We will start by modifying our Projectile Bullet object.

   Locate and Open the **ProjectileBullet** Blueprint, inside the ProjectileWeapon folder.

9. Go into Viewport mode inside the Blueprint Editor.

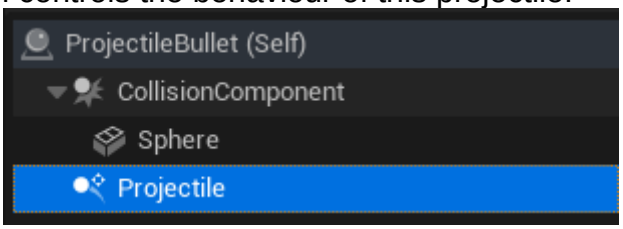


Step 2: Customising the Projectile Settings and their Collisions

1. Start by scaling down the size of the projectile.
   We can scale the collision component or the Sphere, but the Collision Component is the Root, and the Sphere Mesh is the child, so therefore if we scale the Root, then we scale BOTH the Sphere Mesh and the Collision, whereas scaling the Sphere Mesh will leave the Collision Component its original size.

   Use the details view to change the scale to slightly smaller than the default, on the correct component.

2. This blueprint contains a *Projectile Movement Component* called **Projectile;** this is the ActorComponent which controls the behaviour of this projectile.



   Change the settings on this component via the details view, so that *Should Bounce* is False (unticked).

3. Compile and Save the Blueprint. Test your Projectile Fire.

   The bullets should no longer be bouncing, but the size will remain unchanged. This is because when we are spawning the Bullets, it is the spawn process which oversees the scale definition.

4. Open the **ProjectileWeapon** Blueprint, and locate the **FireWeapon** Event.

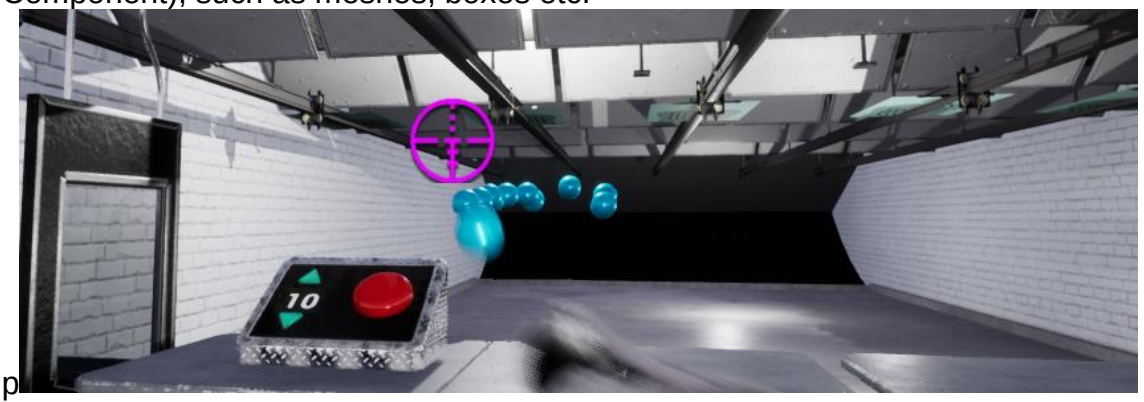
   You should see the Spawn Logic, where the transform is set for the spawned bullet.

5. Set the scale on this node to **2.0 across all three axes,** to make sure this has an effect.

6. Compile, Save and run the application and our Bullets should now be a different size – this is a key point to remember – **if it is placed, the scale comes from the Blueprint settings, if it is Spawned, the spawn process controls this.**

7. Depending on where you placed your targets, you may not be getting collisions on the Targets themselves.

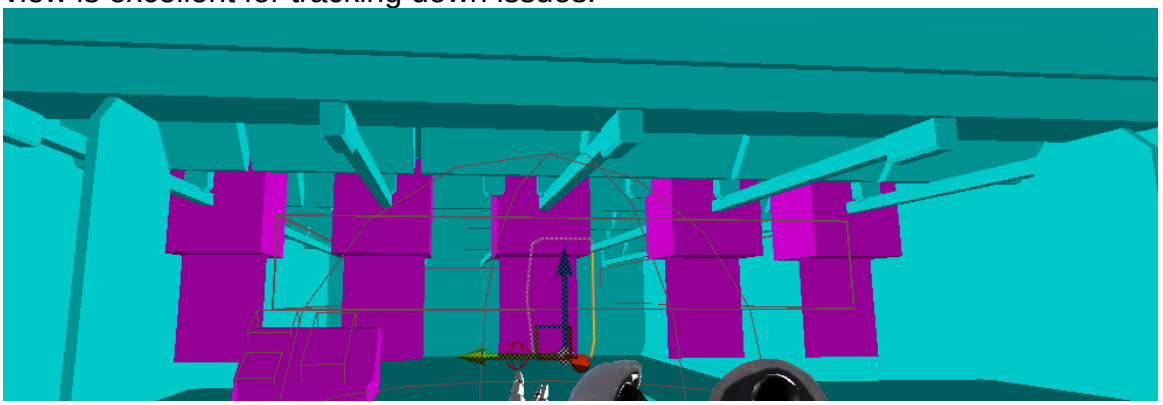
   This is down to our collision settings on any object which is collidable (a Primitive Component), such as meshes, boxes etc.



p

   To resolve this, run the game and **press F8 to eject from the character**. From the View Mode menu in the viewport (which is likely displaying Lit) select the **Player Collision** mode.

   You can also press **Alt + C** to toggle the optional collision show mode as well. This debug view is excellent for tracking down issues:

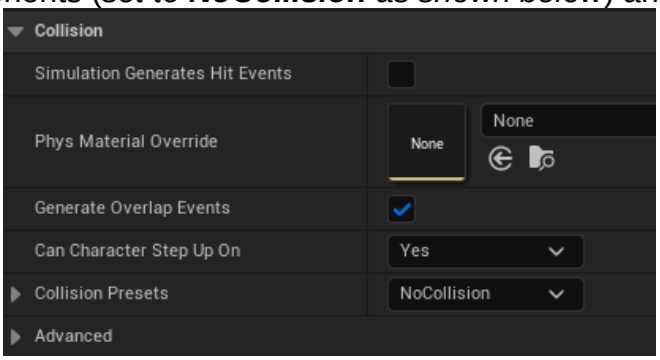


   In this case, we can see unnecessary collision objects on the Targets themselves, but also, as a wireframe, on our Obstacle Spawn Volume.

   To correct this, we need to change the collision settings – these are on the Primitive Components on the Mesh.

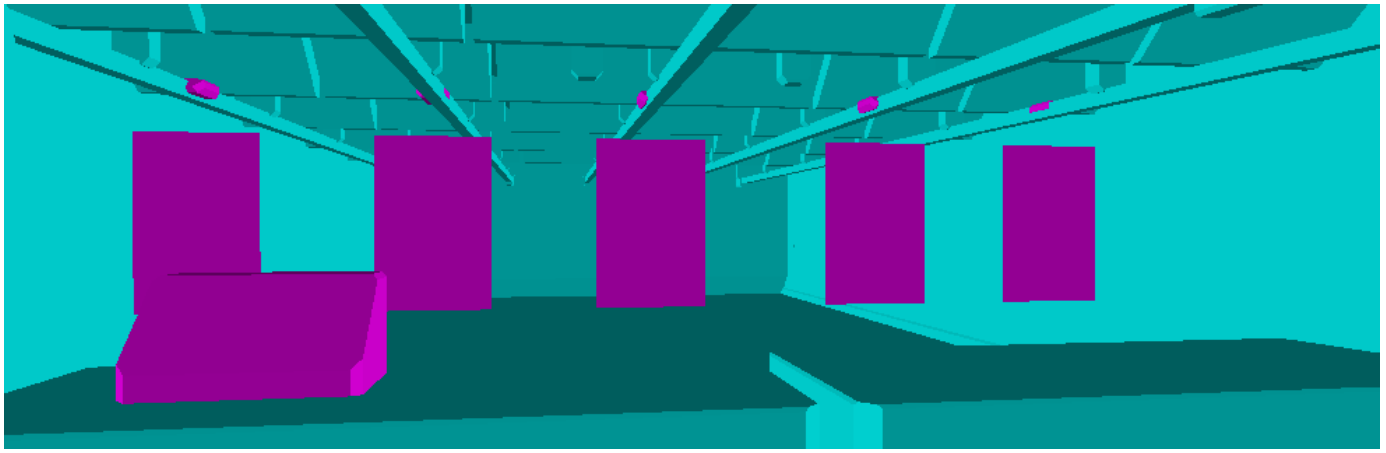
   Make sure the game is not playing when you make these changes.

8. For each Blueprint below, open them, change the settings on the following Blueprints and components (*set to **NoCollision** as shown below*) and compile and save all Blueprints:



- **DynamicTarget_BP** – Box
- **RailPaperTarget_BP** – RailClip.

This will make our collision much more accurate and avoid false collisions with elements other than the targets



9. Test this out and you should now hit the objects (as your collisions should be like the above) – from Tutorial 16, the Projectiles will be destroyed when they hit a target.

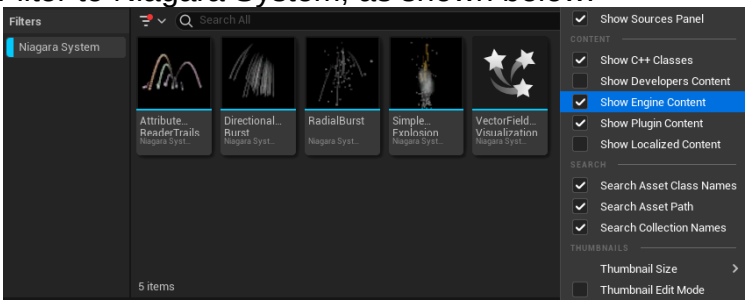
Step 3: Player Feedback and a Damage Model

1. Next, we will add an effect to give the player feedback when a collision has occurred.

   For this we will add a **diegetic effect** – these are in-game feedback that are not typical UI elements, but designed to feel part of the virtual world – example of this include fire, smoke, grenade flashes, blood trails, recoil animations etc.

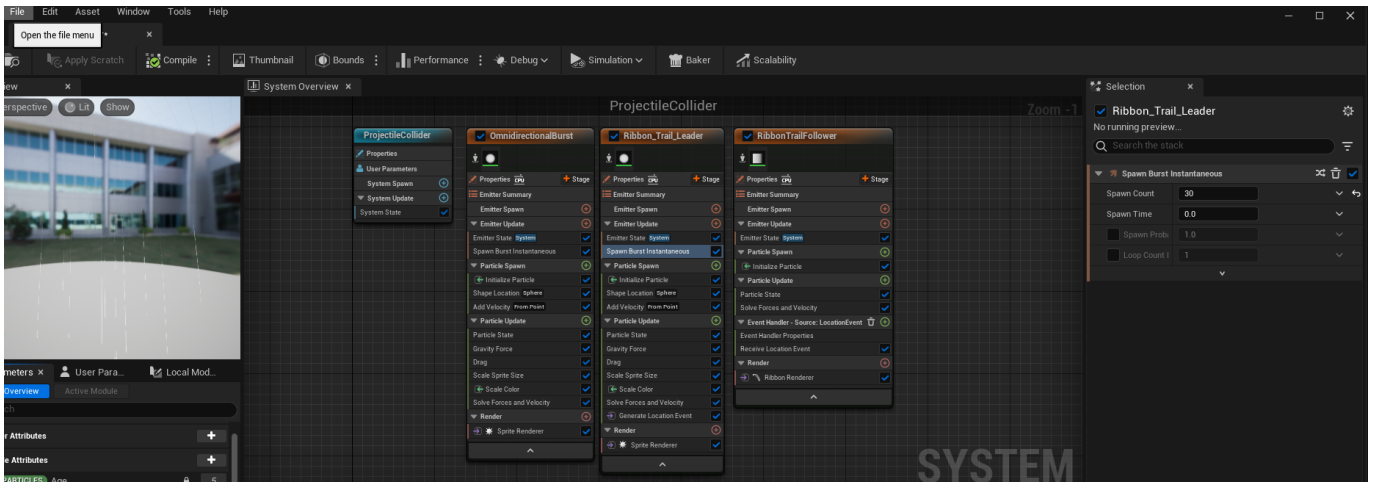
   In this case, we will use a simple particle effect.
   There aren't any included in the template, so we need to adapt one from the engine itself.

   Choose the Settings in the top-right in the content browser and enable 'Show Engine Content'. Set the Filter to Niagara System, as shown below:



2. Take a copy of the **RadialBurst** Emitter, this by dragging the file into the **GameBlueprints** folder of Content and selecting *Copy Here*.

3. Rename this file to **P_ProjectileCollider**. We will trigger this effect when the Bullet hits the target.
4. Double click on this file to open it in the **Niagara Editor**



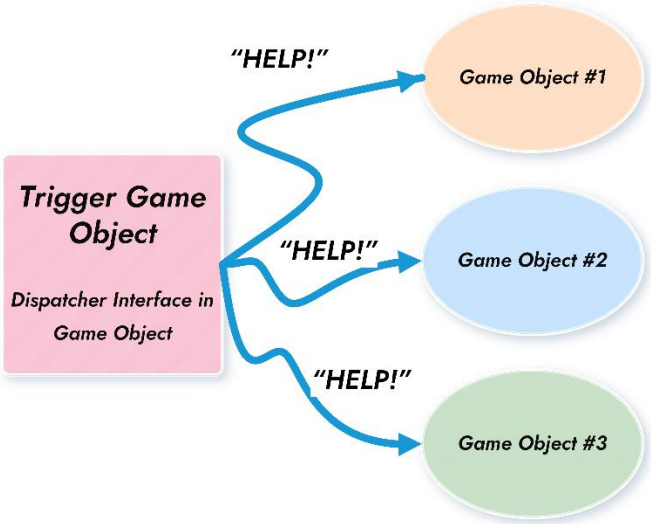
5. Make the effect more pronounced, by selecting the Ribbon_Trail_Leader's Spawn Burst Instantaneous property, and in the details view, increase the Spawn count to at least 30 – you should see this update in the preview.

6. Compile, Save and then you can close the Niagara editor.

7. Next, we are going to use another type of Event Dispatcher to handle this – in the last tutorial (Tutorial 16) we used **open-ear** dispatchers, so that we could inform our Target when it had been hit by an object.

This time, we are going to use **open-mouth** or Broadcast dispatchers.

Basically, when an event occurs a message is broadcast from the Blueprint to all interested parties.



*Rather than setting this up ourselves, we'll use the **UE5 Damage system.** You can learn about the system in detail, with the following resources.*

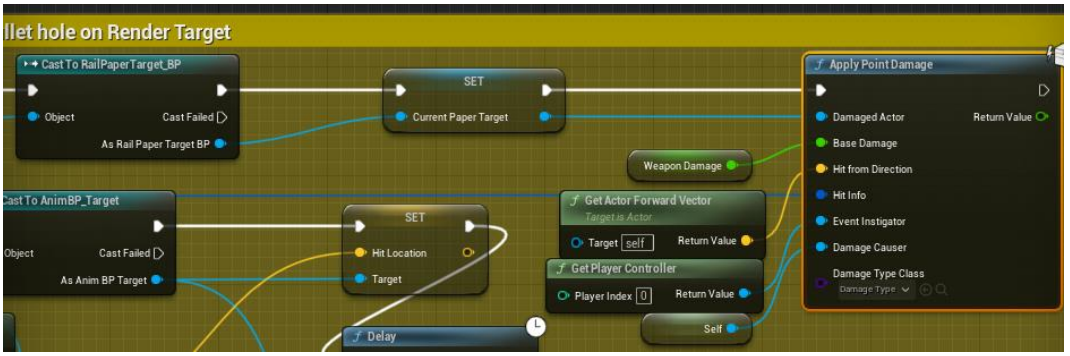
Damage | Unreal Engine 5.2 Documentation
https://www.youtube.com/watch?v=8RuMckVAA4c
https://www.youtube.com/watch?v=5Xdnx3wPNLo

We can call **ApplyDamage** to broadcast a damage event, specifically to the object we have damaged. This then is received by our Actor which has been damaged, to then be handled.

8. Modify the ProjectileBullet's **TargetResponseToBullet** event, so that after we determine it is the **RailPaperTarget_BP** we have hit, we notify it that it has been Hit.

Remove the direct call to **OnTargetHit.**

9. Replace it with the following:



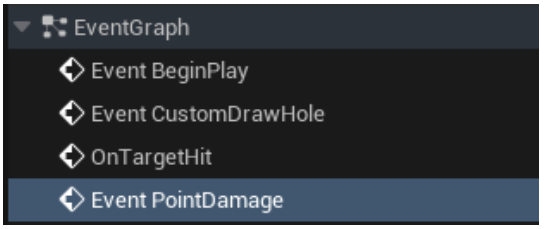
This event takes a lot of parameters to give us a rich set of information about the damage inflicted.
NOTE: The Hit Info is wired to the Hit parameter that is passed as the input to the Macro.

10. Save and Compile the Blueprint.

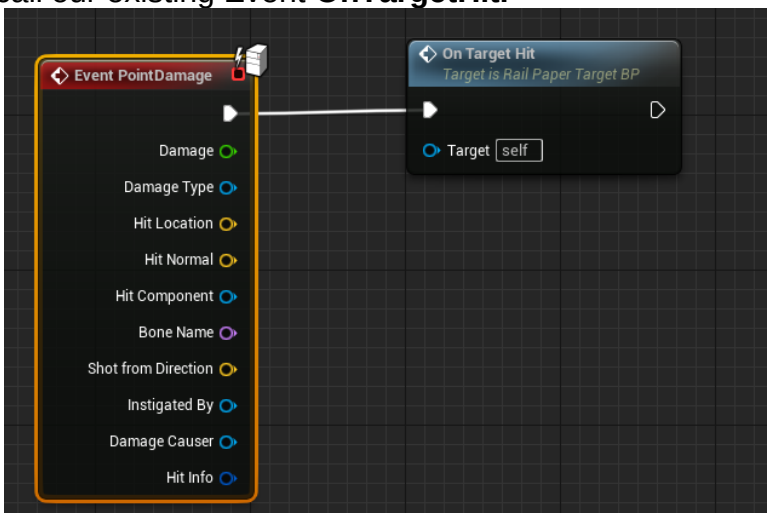
In this case, we will use the very-specific type of damage – **PointDamage** – this tells us where the damage occurred, how much damage was inflicted and from what direction, what caused the damage (this bullet) and who triggered the damage (the player who fired the bullet).

We can even classify the type of damage (*e.g., fire damage, or spell damage etc*). We'll just use the default type of damage (Called Damage Type).

11. Open **RailPaperTarget_BP**. We now need to handle the receiver side. From the override functions list, choose to override the receiver side – this is the function **PointDamage**.



12. Get this event to call our existing Event **OnTargetHit.**

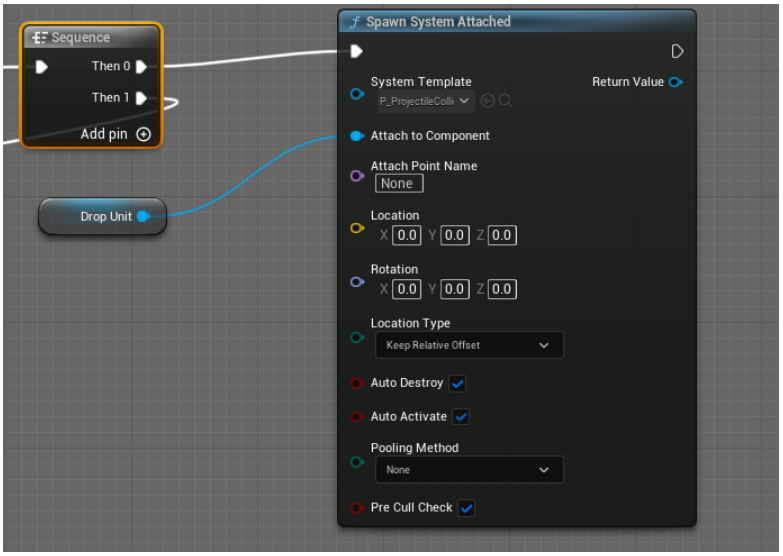


Therefore, both **OnTargetHit** and **PointDamage** will do the same thing.

Notice that PointDamage receives all the information we have attached to the ApplyPointDamage message but also calculates some additional information – the more data passed via events, the richer our gameplay experience can be as we can factor this information into our gameplay logic – **we are not exploiting this in the tutorial – but it is something you should look into for the assignment.**.

13. After the ++ node of **OnTargetHit**, add a **sequence** node and then connect up the Cast To ProjectileCharacter to **Then 1.**

14. To **Then 0**, we will add our **P_ProjectileCollider** Niagara Particle Effect, by adding the following code.



This attachs the effect to the Drop Unit of our Target. We are not using the location here, but we could pin-point the exact location of the collision from our PointDamage event information (again for that richer experience).

15. Compile and Save the Blueprint.

16. Run the game and test that the collisions are occuring and our Particle System reflects that a hit occurred.

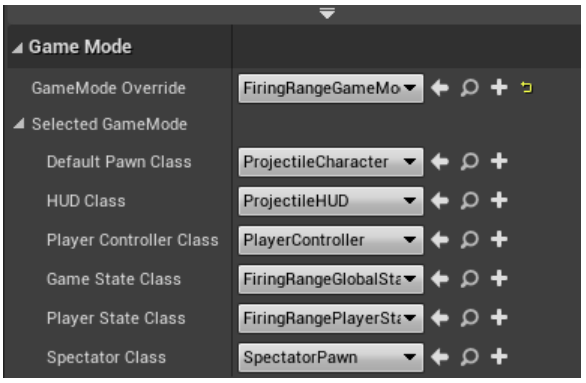
Incidental effects and diegesis are important parts of modern games – player feedback needs to be much more visual than just statistical data.

Task 2: Best Practices of User Interface Handling in UE5

In this task, we will add both a 2D and 3D User Interface to our Game. We want to make sure this is handled correctly and not tied to a specific Pawn, so we will use the gameplay framework to help us.

Step 1: Associating data and our UMG Widget with our HUD

1. We'll add the User Interface via our **ProjectileHUD** class – this should be set in the GameMode as our HUD Class:



   You can find a description of this class here:
   [User Interfaces and HUDs in Unreal Engine | Unreal Engine 5.2 Documentation]

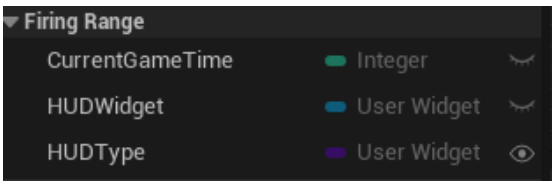
2. We'll be using **UMG (Unreal Motion Graphics** for this system. This is one of two ways of handling User Interfaces – the other is the older system known as Slate – this is the UI system that is used to create the UE5 editor. We are already using Slate to draw the Crosshair Reticule, but it is much harder to build UIs in comparison to UMG – the two can work together very nicely.

   Go to the **ProjectileWeapon/HUD** folder in the content browser and inside that, right-click and select **User Interface -> Create Widget Blueprint.** Widgets are UMG UI elements – they can represent a single UI element such as a waypoint, or an entire screen of interface such as an inventory screen, pause screen, or menu screen. We are going to create a widget to act as the permanent HUD of the player. Therefore, rename this widget: **Widget_ProjectileHUD.**

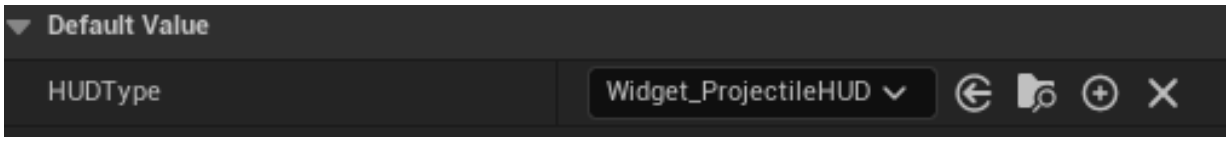
3. Before we do anything with this UI, we need to set it up. We'll start by sorting out our HUD class to create the HUD and add it to the viewport, as well as some useful data.
   Open **ProjectileHUD**.

4. Add an **Integer** variable as a class member called **CurrentGameTime**. Remember you need to compile before we can set defaults. Set the default to 0.

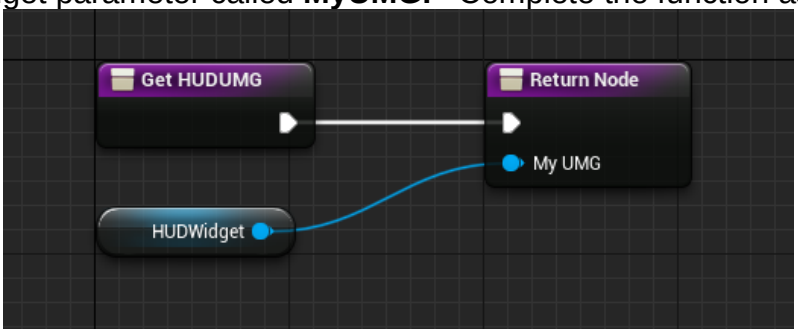
5. Next, add an object reference to a UserWidget called **HUDWidget** and a class reference to a UserWidget called **HUDType.** Make the last one Instance Editable. Put them all in our **FiringRange** category.



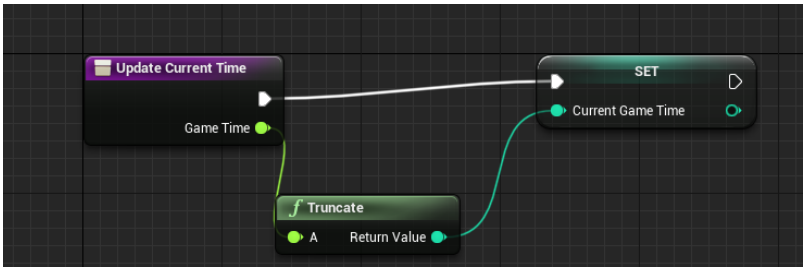
6. We need to associate our UMG Widget with our HUD, so select HUDType and under details, set its Default Value to our **Widget_ProjectileHUD:**



7. Create a new function called **GetHUDUMG**. Under the details view, Press Outputs + and make a UserWidget parameter called **MyUMG.** Complete the function as follows:



8. Create a new function called **UpdateCurrentTime.** Add **an input** which is a float and name it **GameTime** and complete the function as follows:



The truncate node should be created for you automatically if you try and drag the game time into our Setter for our class member variable.

9. Put each function into the **FiringRange** category and then Compile and Save the changes.

Step 2: Accessing our Gameplay Framework Classes

1. To make our lives much easier, we are going to use another new concept – a **Blueprint Function Library**.

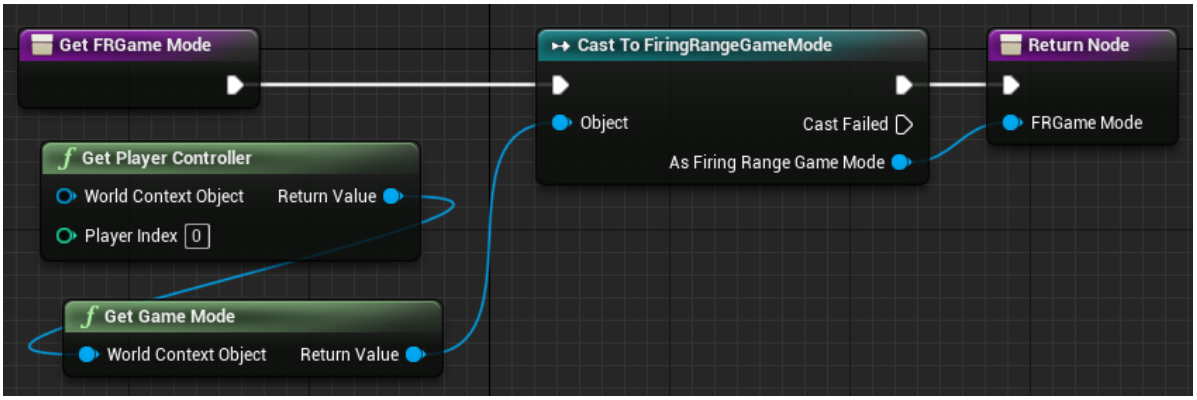
   As the name suggests, they are there to provide functions that are useful for Blueprints – they are a grouping of functions, much like our utility classes in the SEG Engine, which help simplify certain calculations – in this case, getting hold of our Gameplay Framework objects (like the ContextProvider helps us with).

   Go to the **FiringRangeGame** folder in the content browser and right-click and select Blueprints -> Blueprint Function Library – name this library **BFL_FiringRangeHelpers.** Double click and open it.

2. Using Add New -> Function, create five functions called:
   - **GetFRGameMode**
   - **GetFRGlobalGameState**
   - **GetFRPlayerGameState**
   - **GetFRProjectileHUD**
   - **GetFRRayCastHUD**

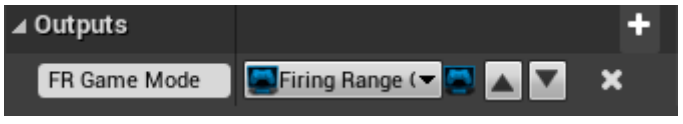
Put them all in the FiringRange category. Open each function up and within each, right-click and create type 'Add Return' and insert a **Return Node**.

We'll complete them in order. Add the following nodes to **GetFRGameMode:**



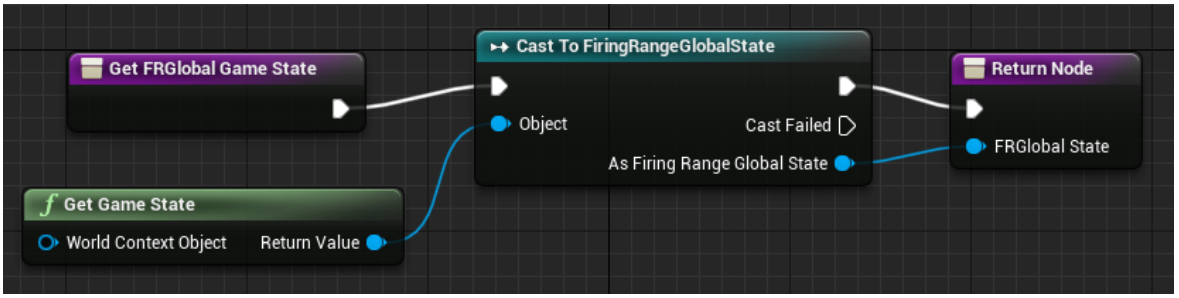
We get the active player to then get the game mode and downcast it to the correct type.

To wire up the Return Node, drag the Blue pin from As Firing Range Game Mode and drop it into the Return Node. Then select the Return Node and in the details view, change the Output name of this parameter to FR Game Mode – this is the easiest way of adding return nodes, since it detects the Actor's class automatically.

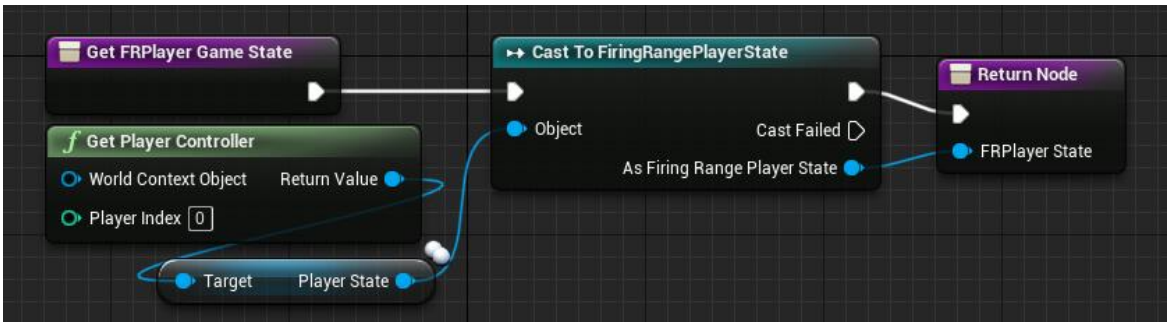


We are going to use this approach with the creation of all our Helper functions.

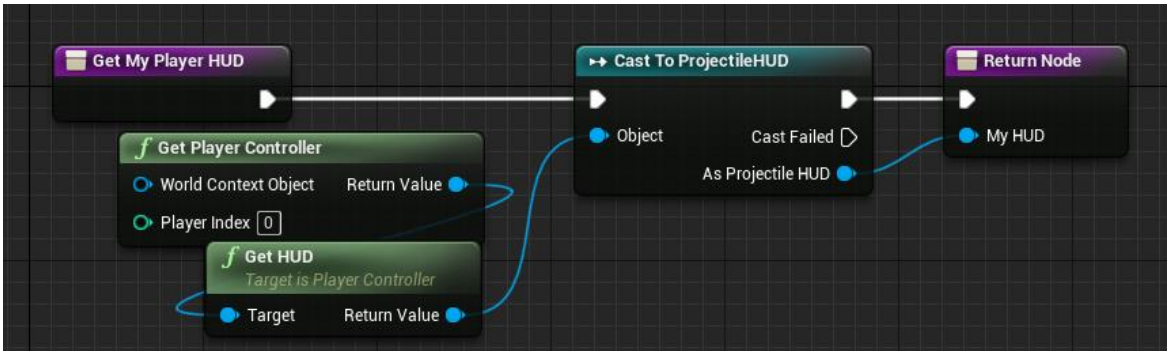
3. Complete **GetFRGlobalGameState** with the following:



4. Complete **GetFRPlayerGameState** with the following:



5. Complete **GetFRProjectileHUD** with the following:



In all cases, we are violating the Liskov Substitution Principle (LSP), which is very common in UE5, because we are down casting to our specific classes – but these helper functions will **make it much easier to access key objects of our game.**

Many commercial UE5 games take this approach.

At least if we change the game mode or game state class, we only need to change the functions here and not in multiple Blueprints (**there is less coupling**).

HUD is less elegant, and we will see the issues with this later – **again, you should be eliminating the need for two HUD classes in your Refactor!**

6. You should be able to complete GetFRRayCastHUD by yourselves using the answer to Step 5 as the basis.

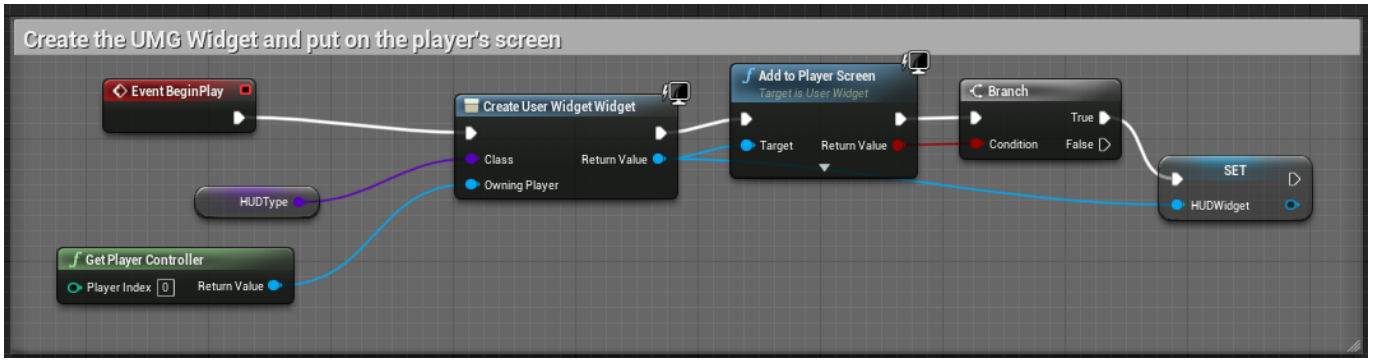
7. **Compile and Save this Blueprint.**

Step 3: Creating the UI and Binding Game State to our HUD/UMG classes

So, we have set up our HUD class and our Helper functions, now we'll use them to report our current game state to the User. We will add this HUD for the Projectile Character.

1. Open **ProjectileHUD (which is the HUD designed for the Projectile Character)** and override the **BeginPlay** event.

Wire this up as follows:



This uses our **HUDType** variable to create our UMG Widget – this way, we can create different HUDs which we can change by assigning a different class to the variable – we then add it to the player's screen (which is the Viewport) and finally store the Widget we created locally.

2. Next, we will add a way of getting the playing time for our game – we will let the Game Mode oversee this code. Locate and open **FiringRangeGameMode.** Start by adding a Float variable called **RunningTime** to this class.

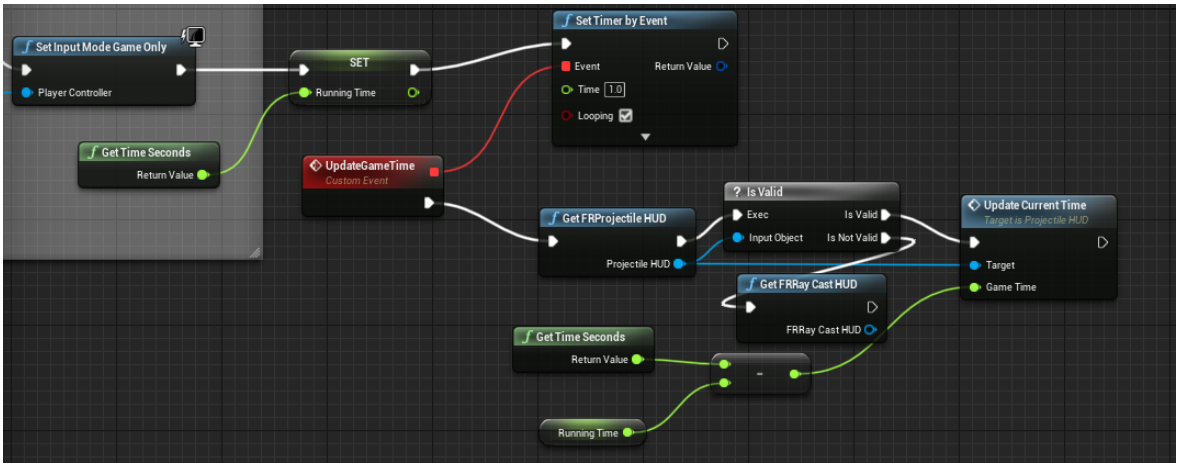
3. Find the BeginPlay event in the Event Graph and after we set the input mode to Game Only, right click and add a new custom event called **UpdateGameTime.**

We are going to store the current running time of the game in our Running Time variable at the point this delay runs out and use this to calculate the difference between this time and the actual game time to give us the duration which we have been active in the Firing Range.

We will then update the HUD with this value using our UpdateCurrentTime function we set up in the last task.

Rather than updating the time every frame, we will instead do this on a per second basis since we only need to be accurate to a second.

4. Therefore, complete the BeginPlay graph with the following extension after the **SetInputMode Game Only** node.



Notice that our Blueprint Function Library Get FR Projectile HUD node made this trivial to access our exact Blueprint (Projectile HUD).

If our HUD node is Null/None, then it means we have a different HUD (e.g., the Ray Cast HUD).

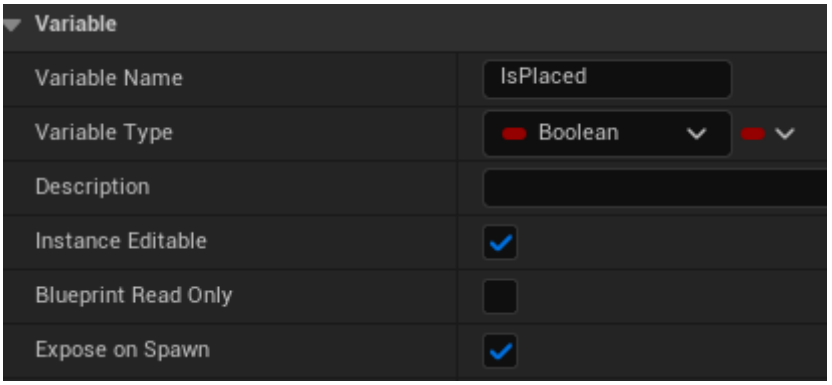
If it is the Projectile HUD then we call Update Current Time passing in the time calculated as the current time minus the time when the game started (e.g., if time is 40 seconds and but Running Time was 5 seconds at the start then the game time is 35 seconds).

We cannot wire up FRRayCastHUD since it doesn't currently have an UpdateCurrentTime function – this will be a task for you to complete at the end of the tutorial.

5. We also need to update our code so that **MaxHits** is correctly updated based on the objects that are placed in the Level (*or set to an absolute value*).

   To do this, we need to change it so that our IsPlaced Boolean is correctly set.

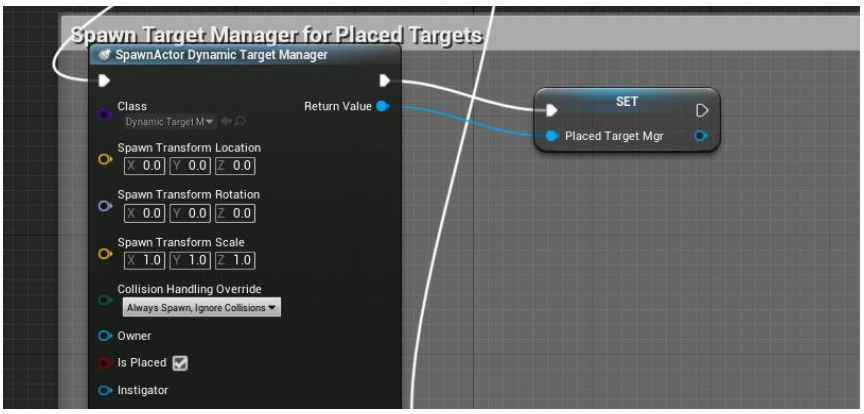
   Open the Blueprint for the **DynamicTargetManager**. Select the IsPlaced variable from the My Blueprint section and then in the Details view, make sure that IsPlaced is set as **Expose on Spawn**:



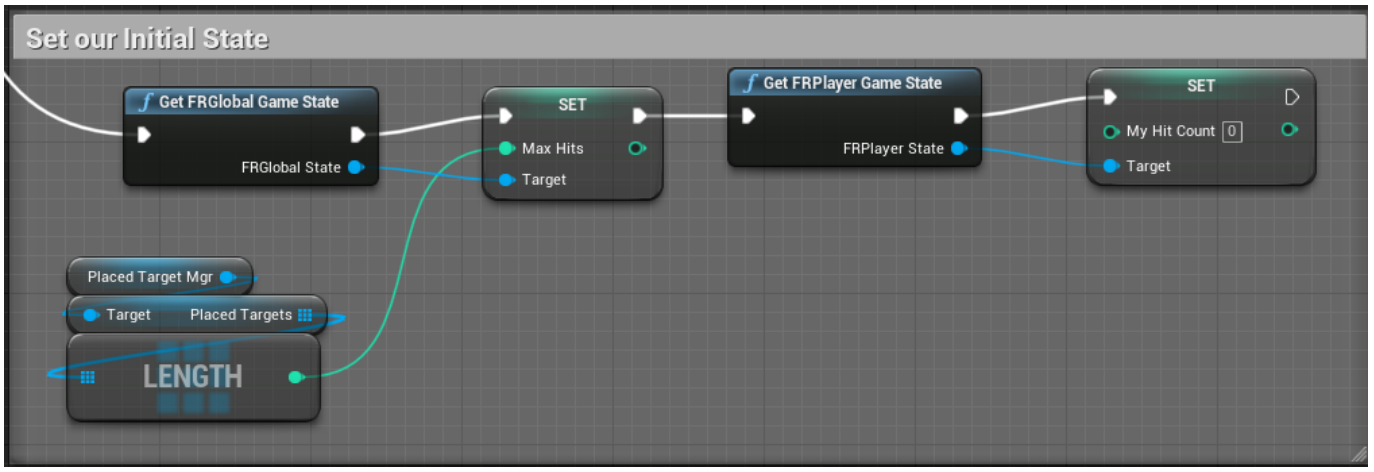
   For this to work correctly it must be **Instance Editable** as well.

6. Compile and Save the Blueprint.

7. Back in our FiringRangeGameMode blueprint, locate the logic where the Spawn Dynamic Target Manager logic is working – remove the existing code and replace it with the following – notice that **IsPlaced** is now a setting on the Spawn Node and make sure it is set to True.



8. Because of our Blueprint Function Library, we no longer need the Macros we used in Tutorial 08, so we can replace the Macros with our Helper functions instead in **Then 2**:



9. Now we can complete our UI Widget, so it is getting the data from our two different Game States and receive the Updated Time from our Game Mode.

   This is where we use Binding, and this was the purpose of all of this helper code – we can now easily access all of these objects to Bind the state to the UI.

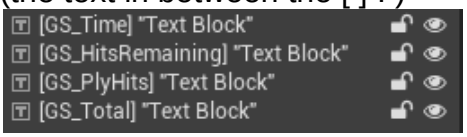
   Open **Widget_ProjectileHUD** and go into the Design view. This is where we will construct our User Interface – we will keep it text-based, but the Binding is very similar for the other component types – you can use these concepts to make a more sophisticated UI.

   Make sure the UI is set to Fill Screen:

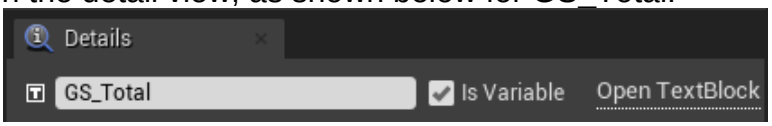
   Drag eight new Text Controls onto the Canvas – we will position them later.

Start by setting the Text of the first four, by selecting them from the Hierarchy view and changing the Content -> Text part in the Details view – the Text to use is in string in the " " in the below screen shot:

10. Because these are just hard coded text, we won't worry about naming them, but for nodes we want to Bind it is worth naming them correctly. For the next four Text elements, change their names to the following (the text in between the [ ] :)



This name is set in the detail view, as shown below for GS_Total:

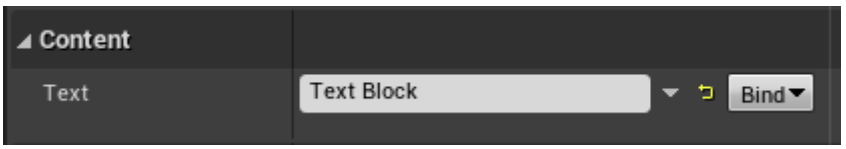


Make them variables as well by setting **Is Variable** to true.

11. Now we have our Data, we can lay out the User Interface – I have positioned mine as following, but you can organise it how you want, but we want to group a variable next to its corresponding label:

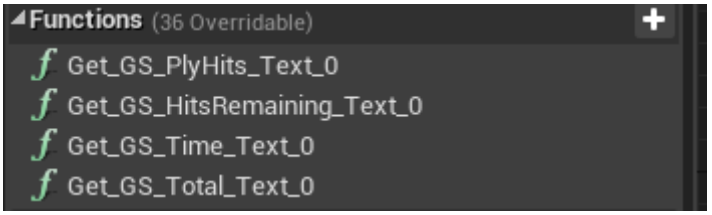
GS_Time goes next to Current Time, **GS_HitsRemaining** next to Hits Remaining, **GS_Total** next to Total Hits and **GS_PlyHits** next to Player Hits – these are the four pieces of state we are conveying to our player.

12. The text "Text Block" will never be displayed to the user, because we are going to replace this text with our state data instead. To do this, for each of our **GS_** text nodes, go into the Details view and select the Bind button next to the Text content and choose **Create Binding**:



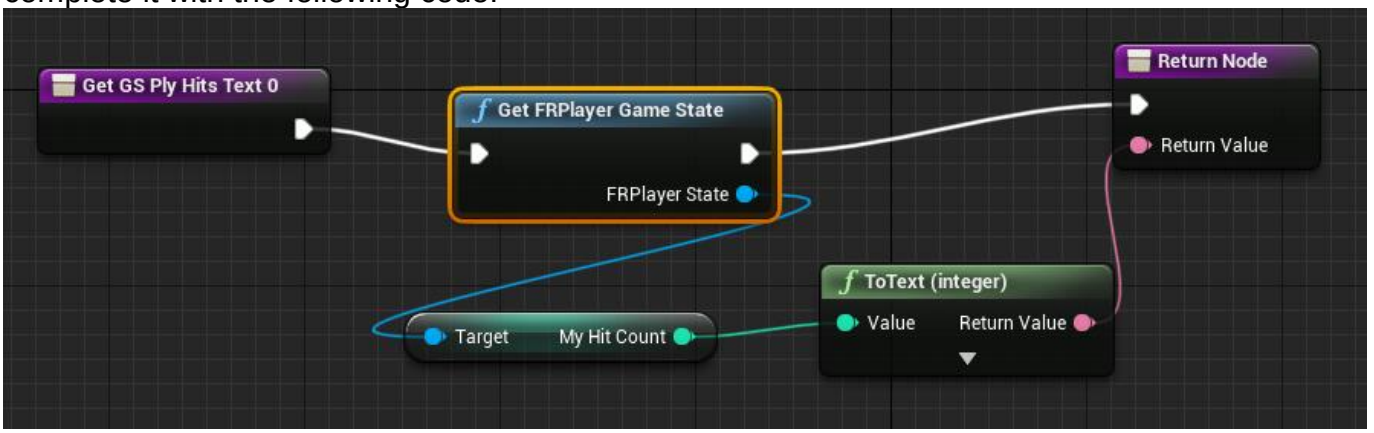
This will generate a function in the Blueprint Graph – this will take you to the Graph view, so click the **Designer** tab in the top-right to return to the UI designer, so you can complete this step for all four Text Blocks.

13. Finally, we will complete the code driving our functions – because we named our Text nodes, the function names generated are sensible – they should be the same (or very similar to the following) in the Graph view:



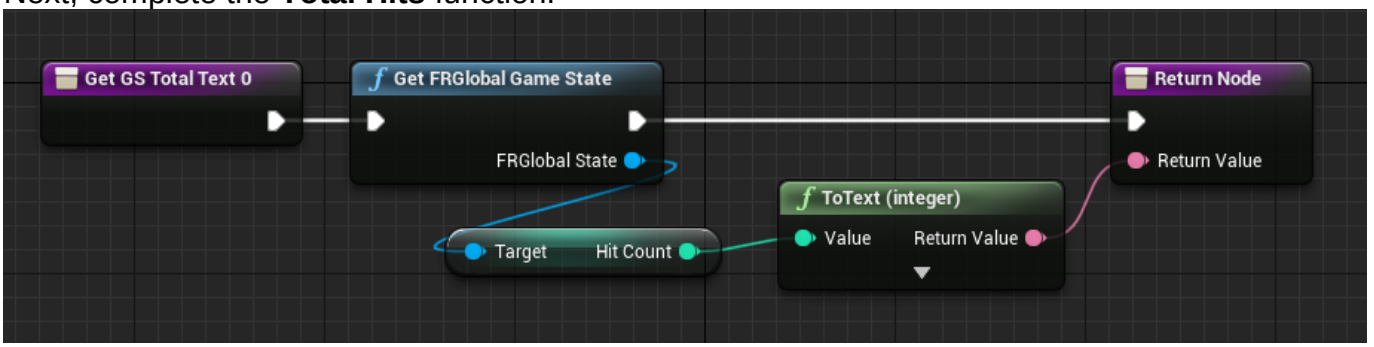
We are using an UE5 UMG concept called **Function Binding** – these functions have a specific job which is to return the text that we want to display on the corresponding UI element it is bound to – UE5 will take care of the rest for us!

Open the Function for the **Player Hits** state (should be obvious from the names!) and complete it with the following code:

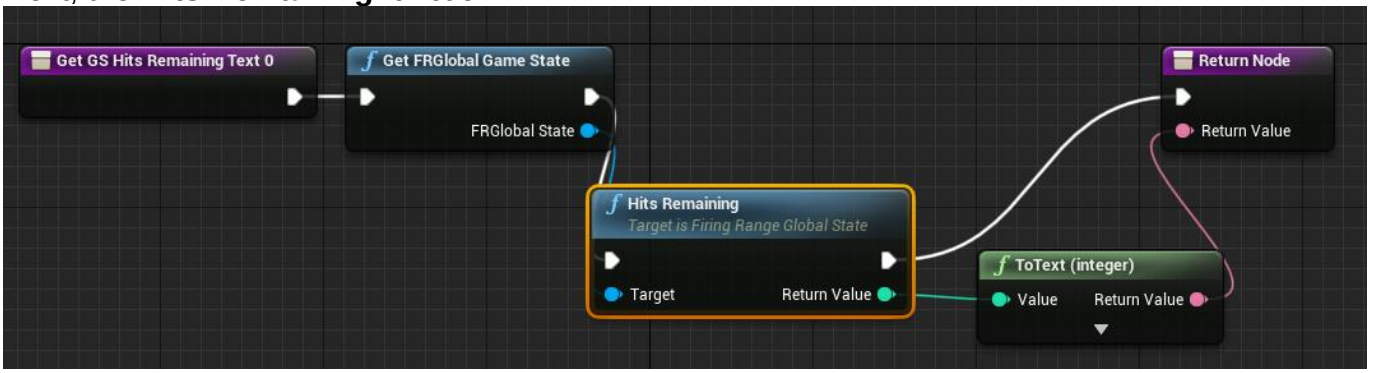


If you drag My Hit Count directly onto the Return node, this should automatically create the ToText node for you.

14. Next, complete the **Total Hits** function:

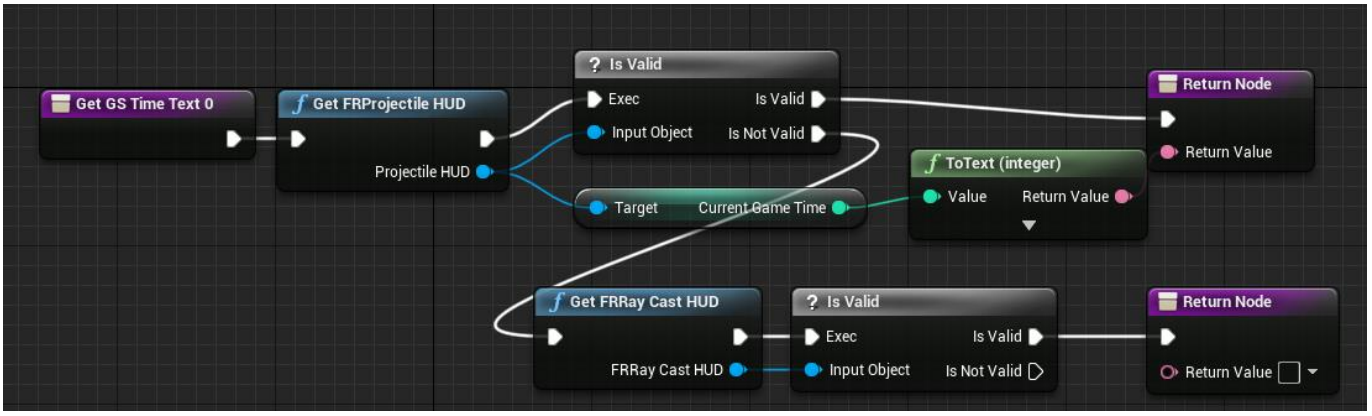


15. Next, the **Hits Remaining** function:



In all three cases, we have used our Helper functions to get the relevant State class and access the data that is specific to that class – it makes our UI creation so much easier doing it this way!

16. Finally, complete the **Current Time**: function with the following:

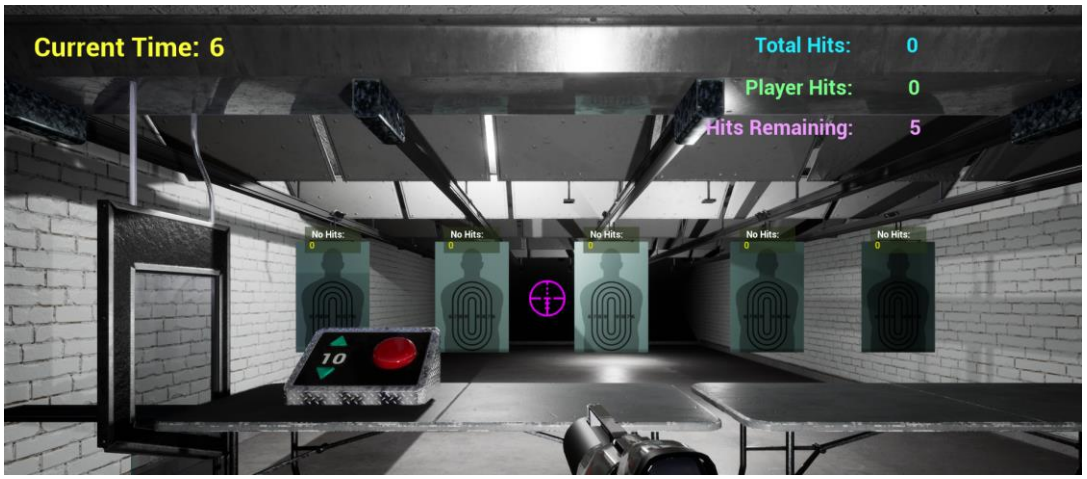


This example is using two layers of communication – our Game Mode is updating the HUD with the Current Time (as we saw in our earlier steps) and then we are accessing the HUD to update the UI – this is better than directly accessing the Game Mode, because it means we can re-use these classes with a different Game mode! Again, we handle when the HUD is not the Projectile HUD by checking for RayCast HUD, which again is incomplete, so we cannot get the current time from that yet. If this code looks clunky, it is!

Remember, our refactor should have one character and therefore one HUD so that will remove this clunkiness.

17. Compile and Save the Blueprint.

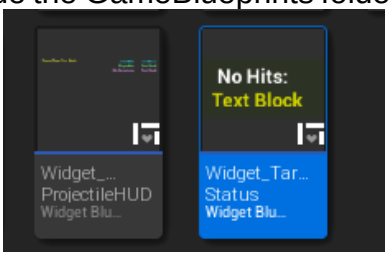
Run the game and you should have a UI which begins counting the time after the delay and the number of hits remaining is the number of targets in your level – damaging a target should inflict a change in both the global hit count and the player hit count – for this game those two things mean the same, but in a multiplayer game (e.g., co-op), we would a have a player total and a combined total.



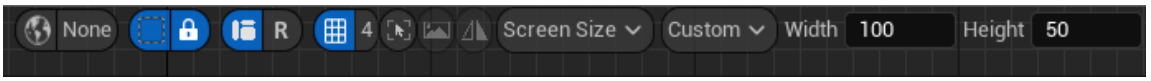
Step 4: 3D Widgets and Contextual Information

In the previous tasks, we have set up a Full Screen User Interface for the player, via the HUD Class and a UMG Widget – this is the best way of handling permanent interfaces, but there are a lots of cases where we use UI in context on objects in the game – for this we can also use UMG Widgets, but this time we associate them with the Gameplay Actors we are using.

1. Create a new UMG Widget inside the GameBlueprints folder called **Widget_TargetStatus.**



2. Open it in the UMG Editor and from the designer view, set the width and height of this object to 100 wide and 50 high:



3. To the canvas, add in a Border and Two Text Elements. Set the Brush Color of the Border in the appearance section to a dark colour, making sure it is semi-transparent (by setting the Alpha to 0.5 or less).

4. Next, set up the text element – we'll use the same approach as before. Change the Text of one of the elements to **No Hits:** - this is going to be our label. For the other Text elements, change its variable name to **TS_NoHits** and leave the text as the default – this is going to be our dynamic value which changes.

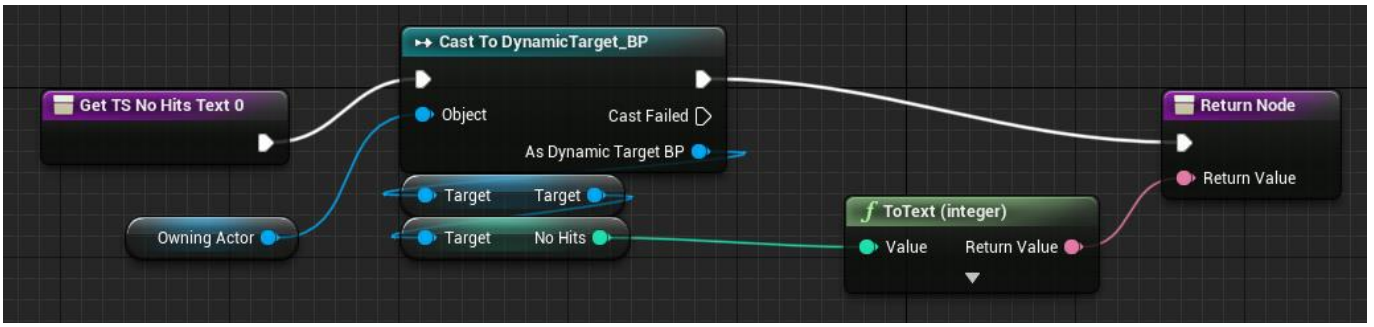
5. Bind the **TS_NoHits** element to a function using the same approach as we took in the previous tasks.

6. Position the text elements inside the Border and format them using font sizes, colours etc so we have a clear display between our label and text – this is what I ended up with:

7. In the Graph view add a new variable to the Blueprint called **OwningActor**, which should be an Object Reference to an Actor.

We going to use this variable to store the Actor whose information we want to be use in our display.

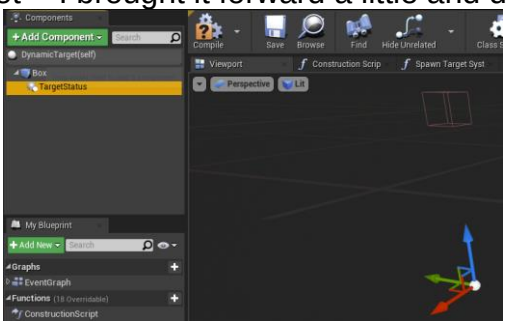
8. The function I created during binding is called **Get_TS_NoHits_Text_0**; yours should be the same or similar.
   Complete your functions with the following nodes.



9. **Compile and Save** the Blueprint.

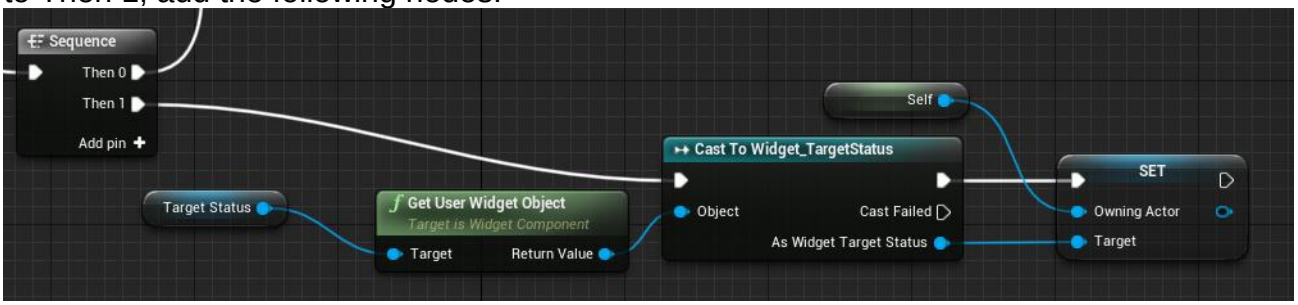
This UMG Widget will therefore be associated with a Dynamic Target (*or Child Class of Dynamic Target*) and will display our No Hits data stored on the RailPaperTarget_BP from within an instance/object of this class.

10. Open the **DynamicTarget_BP** blueprint in the editor. Add a *Widget Component* via the Components panel and attach this to the Box. Name this component **TargetStatus.** In the details view, set the Draw size to X:100, Y:50 and the Space as **Screen** – this will make it a 2D Projected element. In the Viewport, you can adjust its position relative to the Box, which is the top of the target - I brought it forward a little and down a bit.



11. Finally, in the Details view associate this component with our new UI Element by setting the WidgetClass to **Widget_TargetStatus** – this will mean we can then spawn this element.

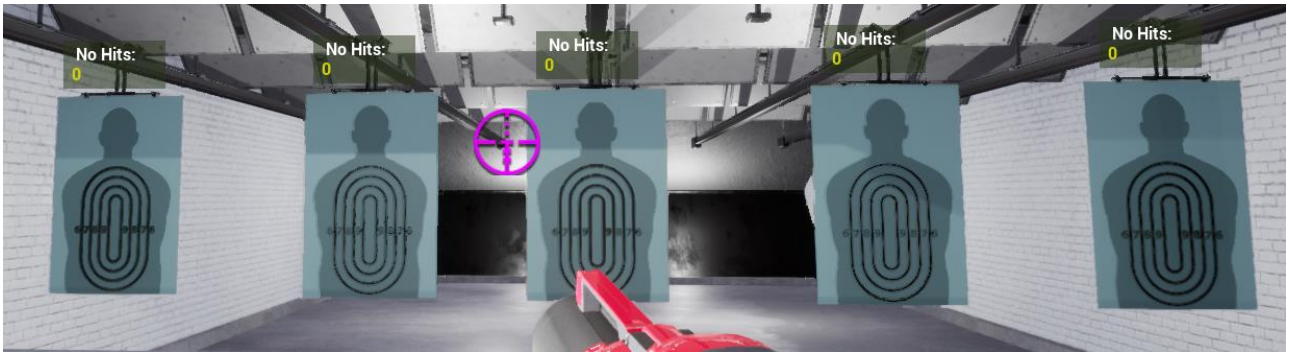
12. To complete the setup, we just need to associate this object with our UI Widget using our OwningActor variable we set up.
    In **BeginPlay**, add a sequence node so that all the existing code is bound to Then 0, and to Then 1, add the following nodes:



We get the UMG Widget from the component, downcast it to our Widget_TargetStatus type so we can access the OwningActor variable. We then set it to a reference to **self**, which is the equivalent of *this* in C++ - it means this particular object (the instance of DynamicTarget_BP, or child class of DynamicTarget_BP) – now this means DynamicTarget_BP is carrying around its own UI element!

13. **Compile and Save the Blueprint.**

14. Test our game, and you should find that we now have UI elements on the target that display the number of hits they have received.



Step 5: Additional Tasks

So, we have a game UI and contextual UIs for our Targets – but only for the ProjectileCharacter

1. Try to apply the concepts we have created here to the RayCastCharacter so that this can also make use of our HUDs.

Task 3: Experimenting with our Code and Moving it into the Coursework Template

So, over the last three tutorials we have created our Target System, worked with the Gameplay Framework to control our Application, used Timelines and raw linear algebra to animate our targets, add in collision detection and used concepts like event dispatchers and now we have two different types of UI in our game!

The remainder of the coursework is over to you!

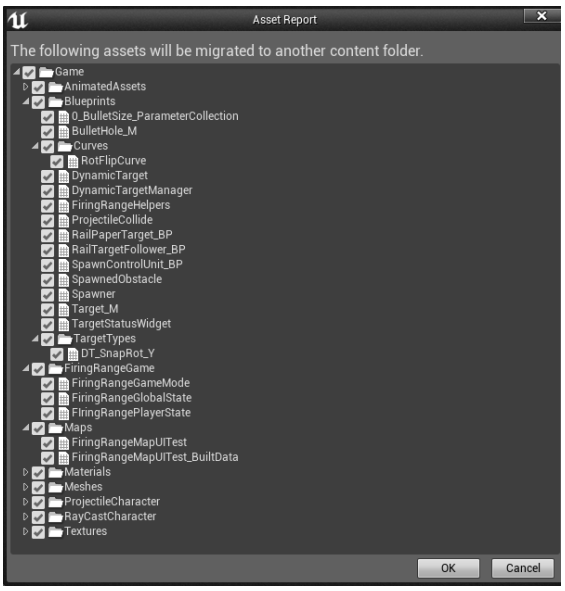
You can use any of the concepts from these tutorials in your solution to help build your game – it is fine to use your completed version of Tutorial 17 as the basis for your coursework.

If you have already worked on a different copy, you can instead use Migration, as covered below.

Step 1: Content Migration into your assignment version

You can migrate content such as Blueprints to another copy of the template by right-clicking on the asset add selecting **Migrate** and choosing the Content folder of the project you want to move them to – this will also copy any assets which those Blueprints are dependent on!

For example, if I migrate the Map we have made in this tutorial:



You can see it is going to copy the game mode, the states, the new Blueprints we have made – notice also that its only the content that is needed by the level – in this case, since we have only used our Rotating Target, it is not going to copy any of the other target types or Curves that we have made.

Make sure you put everything in your level that you want to move across before migrating and remove elements that you want to remain the same from the original project (e.g., this will also replace the two characters with the ones from the tutorial – you can always rename the assets before migrating them to avoid conflicts with any work you have done).

Step 2: Improving the model solution to this tutorial.

Because Migration is so easy, I recommend using a copy of your tutorial version of the code as a way of prototyping stuff before you incorporate it fully into your coursework – **here are some recommended tasks.**

- Experiment with the UI Widget on Dynamic Target to use a mode other than Screen – this will make actual 3D widgets instead of 2D Projection.
- Try and alter the HUD class to contain widgets for Winning/Losing (you will need to create new widgets for this) – see if you can then add a condition to the game mode which will win the game (e.g. if hits remaining becomes 0) and lose the game (e.g. hits below a certain number) and show these to the player when these conditions are met – the game mode is the best place for this type of logic.
- Make a more complex scoring system – the number of hits is not a great approach – see if you can give each target a separate score value and modify the existing logic to score hits in a different manner.
- See if you can dynamically spawn targets using the DynamicTargetManager – we have the skeleton code for this in place and we have seen how to do dynamic spawning.
- See if you can create your own variant of the Control Unit system to do something different in your game – in this example, we use it to spawn obstacles, but you could use it to start the game, or change the difficulty.
- Experiment with different combinations of targets and timelines, you should be able to create a lot of different types of targets and use the delay system to change when they start animating / moving.

Step 3: Continuing the UE5 Tutorials.

The remaining UE5 tutorials (Tutorials 19 – 21) will be using a different code base to try and demonstrate some more of the OO principles that we want to apply in UE5, so will not be directly applied to the coursework code.

However, you should still look at them!

These should help you with things like modelling the weapon switching etc, but that task is down to you – my advice is to start by adding the concepts from ProjectileCharacter/RayCastCharacter added in the last three UE5 tutorials into the RefactorCharacter and see if you can get that class to hold both a RayCast and Projectile Weapon.

For the highest mark, try and make use of the concepts from the lectures – the Strategy Pattern, Aggregation/Composition and Polymorphism with interfaces/abstract classes are a great way of handling this to avoid "clunky" code!

These have the highest scoring value in the Rubrics.